SIMATS ENGINEERING

DEPARTMENT OF INFORMATION TECHNOLOGY

NATURAL LANGUAGE PROCESSING – ASSIGNMENT 5 (CO5)

LANGUAGE GENERATION AND DISCOURSE ANALYSIS

Student Name:Totti Reddy Gayathri

Register Number:192425239

Department / Section:3rd year/BTECH

Faculty Name:Dr.PremKumar

Date of Submission:8-11-2026

PART A – DESIGN, SPECIFICATION AND PSEUDOCODE (25 MARKS)

A1 – Define the Discourse Analyst's World (4 Marks)

A1.1 Position and Responsibilities (1 Mark)

Pipeline: Raw report → sentence/clause segmentation → mention detection → entity and reference resolution → discourse graph → question answering. The discourse analyser receives the text and linguistic annotations. It must preserve the entities, event roles, reference links, uncertainty and discourse relations. It may add structure and links, but must not change the facts stated by the report.

The generation module receives a meaning representation, dialogue/document context and target-language requirements. It plans the order of information and chooses referring expressions and sentence forms. It may change wording, word order and aggregation, but must preserve the intended propositions, participants, time and commitments.

Design issue	Example decision	Settled by
Unit of analysis	Use sentences as initial units; split coordinated clauses only when each expresses an independent event.	Corpus annotation scheme fixes gold unit boundaries; designer chooses operational segmentation if not supplied.
Knowledge source	Use number/person agreement, syntax, recency, grammatical	Designer chooses resolver features; corpus provides gold

	role and entity salience to resolve “It”.	antecedents.
Coherence relation	Label “The suite crashed…” as Explanation/Result relative to installation/build context even without “because”.	Corpus scheme defines relation labels; designer predicts the relation.
Discourse granularity	Use sentence-level EDUs for T, with clause-level splitting for coordinated events in sentence 6.	Corpus scheme defines annotated granularity; designer implements it.
Evaluation measure	Mention-level precision, recall and F1 for reference links.	Designer selects and fixes the measure; corpus supplies gold links.
Genuine ambiguity	Keep two candidates and return AMBIGUOUS if their scores are tied or too close.	Designer sets confidence policy; annotators may mark multiple acceptable readings.

A1.2 Measurement Model (1 Mark)

A markable is an expression that refers to an entity in the discourse, such as a noun phrase, proper name or referential pronoun. A correct resolution is a predicted pair (mention, antecedent) that exactly matches a gold pair. The four gold links in T are the evaluation set.

System	Predicted links	Correct	Gold links	Precision	Recall	F1
X	5	3	4	3/5 = 0.60	3/4 = 0.75	2PR/(P+R) = 0.667
Y	3	3	4	3/3 = 1.00	3/4 = 0.75	2PR/(P+R) = 0.857

Deploy Y for a high-trust incident assistant because it makes fewer unsupported links and has higher F1. It leaves one gold link unresolved, so a clarification or fallback is still needed. Exact-link F1 ignores bridging (“the module” related to a server), near-identity, partial matches and the effect of one wrong link

on a final answer. This can mislead because one wrong antecedent may change the answer even when overall F1 is high.

A1.3 Reference Resolution Procedure (1 Mark)

SIMATS-style pseudocode:

ALGORITHM RESOLVE_PRONOUN(m, discourse_state)
INPUT: pronoun mention m; prior entity mentions and discourse graph
OUTPUT: antecedent, AMBIGUOUS, or NON_REFERENTIAL
1. IF m is pleonastic (weather it / extraposition it) THEN RETURN NON_REFERENTIAL.
2. C ← entities mentioned before m within the current document/paragraph.
3. Remove candidates that fail person or number agreement.
4. Remove candidates that violate syntactic constraints (for example, binding constraints or impossible grammatical role).
5. Score each remaining candidate:
   score = 4×grammatical_role + 3×recency + 2×repetition + 1×parallelism.
   grammatical_role: subject=2, object=1, other=0; recency decreases with sentence distance.
6. Try candidates in order: agreement-compatible subject, recent object, repeated entity, other compatible entity.
7. IF C is empty THEN RETURN UNRESOLVED.
8. IF top two scores tie or differ by less than confidence margin THEN RETURN AMBIGUOUS.
9. RETURN highest-scoring candidate and update its salience.

Fallbacks: empty set → unresolved and request more context; tie → preserve alternatives / ask clarification; non-referential “It is raining” or “It is said” → do not create an entity link.

A1.4 Resolver Trace (1 Mark)

Scoring convention for this hand trace: subject role=4, object role=2, other=1; recency bonus=3 for immediately previous sentence, 2 for two sentences back, 1 for older; repeated lexical mention bonus=2. Agreement and syntax are hard filters. Scores are illustrative and consistently used for this trace.

Sentence / mention	Candidate set after filters and scores	Chosen antecedent	State carried forward
1. The department bought a new server…	No pronoun; entities: department, server_1, lab.	—	server_1 introduced; subject/event context active.

2. It replaced an old machine…	server_1: subject=4 + recency=3 + repeat/context=2 = 9; department: person/number mismatch for It in intended reading / low compatibility.	server_1	old_machine_1 introduced; server_1 remains salient.
3. Ravi configured the server…	No pronoun; Ravi introduced; “the server” links to server_1 by definite description and entity identity.	—	Ravi and server_1 active.
4. He … installed … on it.	He: Ravi=9; department=older and weak subject candidate. it: server_1=9; suite not yet introduced at pronoun position.	He→Ravi; it→server_1	suite_1 introduced by “the compiler suite” only in next sentence? Actually sentence 4 introduces suite_1 as object; it refers to server_1.
5. The suite crashed…	“The suite” matches compiler_suite_1 introduced in sentence 4; it is a repeated nominal, not pronoun.	—	suite_1 salient; crash event added.
6. Ravi reported the fault… vendor replaced a faulty module.	No pronoun. “the fault” links to crash/fault event; “the vendor” is introduced then repeated; “a faulty module” is a new entity.	—	vendor_1 and module_1 introduced.

The difficult sentence is (2). A simple “nearest preceding noun” rule may choose “the lab” or “the department” depending on token proximity. The designed resolver uses agreement, event plausibility and the verb “replaced”: the entity that can replace an old machine is the newly bought server. The intended gold link is server_1.

A2 – Discourse Units, Discourse Graph and Next-Mention (5 Marks)

A2.1 Discover the Boundaries (1 Mark)

Boundary rules: (1) each numbered sentence begins as a provisional unit; (2) split a sentence at a coordinator only if both sides contain an independent predicate and express separate events; (3) do not split merely because a connective occurs inside a clause; (4) start a new paragraph/topic unit at an explicit paragraph break; (5) a change of main event or topic can create a higher-level discourse unit but does not erase sentence boundaries.

For T, use six elementary discourse units U1–U6, one per numbered sentence. Sentence 6 contains two coordinated events (“Ravi reported…” and “the vendor replaced…”); for clause-level analysis it can be split into U6a and U6b, but the supplied sentence-level convention keeps U6 intact and records two event propositions inside it.

A2.2 Segmentation and Discourse Graph (2 Marks)

Pseudocode:

ALGORITHM BUILD_DISCOURSE_GRAPH(text)
1. SENTENCES ← split text at sentence boundaries.
2. FOR each sentence s:
   a. Split at independent coordinated clauses only when each has its own predicate.
   b. Create EDU node with text, entity mentions and event propositions.
3. FOR each new EDU u:
   a. Detect explicit cue (because, therefore, however, then); map cue to relation.
   b. If no cue, infer relation from event/time/entity evidence; otherwise use UNDECIDED.
   c. Select an open attachment node on the right frontier.
   d. Create edge storing relation, nucleus and satellite.
   e. Update predecessor and successor lists.
4. Close a node when it is no longer on the right frontier.
5. RETURN units and graph.

Relations: Elaboration adds detail to a nucleus; Explanation gives a reason; Result gives an outcome; Contrast presents opposing information; Sequence orders events. A cue phrase is evidence, not a guarantee. With no connective, infer only when event and causal/time evidence support it. An attachment to an earlier unit is allowed only if that unit remains on the right frontier. This is the right-frontier constraint.

Graph for T (sentence-level):

U1: department bought server
  └─ ELABORATION → U2: It replaced old machine that had failed twice
U3: Ravi configured server
  └─ SEQUENCE → U4: He installed compiler suite on it
    └─ RESULT/ELABORATION → U5: suite crashed during build
U5
  └─ EXPLANATION/CONSEQUENCE continuation → U6: Ravi reported fault; vendor replaced module

Top-level structure: [U1–U2] followed by [U3–U4–U5–U6]. Deepest attachment is U5 under the U3–U4 installation sequence (the crash is the consequence of the build/install context). Relations without explicit connectives are marked inferred with confidence, not treated as certain.

A2.3 Next-Mention Information (1 Mark)

Use a backward scan because the next mention is found by looking from the current mention toward later units. Assume entities on the right frontier remain available; an entity not mentioned again has next-distance = ∞ and is not available for future reference unless world knowledge or a later bridging relation reintroduces it. Distance is counted in EDU steps.

Unit / mention	Entity	Next mention	Distance	Available on exit?
U2: It	server_1	U3: the server	1	Yes
U2: old machine	old_machine_1	No later mention in T	∞	No
U3: Ravi	Ravi_1	U4: He	1	Yes
U3: the server	server_1	U4: it	1	Yes
U3: lab	lab_1	No later explicit mention	∞	No

A2.4 Examine the Design (1 Mark)

- A connective in the middle of a sentence does not automatically open a new EDU because it may connect clauses inside one unit. Splitting at every connective creates fragments without independent propositions and can attach relations to the wrong span.
- A forward scan cannot know the next future mention when it reaches the current mention unless it stores and revisits the whole document. It may label an entity unavailable even though it appears later, or incorrectly use past mentions as next mentions.
- A discourse graph records nucleus/satellite structure and relations. A summariser can select nuclei and important satellites instead of treating all six sentences as equally important items in a flat list.

A3 – Generator with Memory (5 Marks)

A3.1 Information Needed (1 Mark)

The generator must remember: (1) a focus list with at most k=2 active entities; (2) a descriptor for each entity (last surface form, grammatical number, gender and entity type); (3) an aggregation buffer holding propositions that can be expressed in one sentence; (4) a given/new flag showing whether the reader has already been introduced to an entity.

Data structure	Contents	Update condition
Focus list	At most two entity IDs, ordered by salience.	Add when mentioned; move to front after use; evict lowest-salience entity if full.
Realisation descriptor	Last form, number, gender, semantic type.	Update after choosing pronoun, definite description or name.
Aggregation buffer	Compatible propositions waiting to be verbalised together.	Flush when relation is incompatible, sentence limit reached or topic changes.
Given/new flag	NEW before first mention; GIVEN after successful introduction.	Change to GIVEN after first referring expression.

A3.2 Referring Expression and Aggregation Pseudocode (2 Marks)

ALGORITHM CHOOSE_FORM(entity e, focus F)

1. IF e is NEW THEN RETURN “a/an + descriptive noun phrase”; mark GIVEN.

2. IF e is in F AND no other focused entity has same gender/number AND antecedent is unambiguous THEN RETURN pronoun.

3. IF e is in F but a same-gender/number competitor exists THEN RETURN definite description or full name.

4. IF e is not in F and |F| < k THEN RETURN definite description; add e to F.

5. IF e is not in F and |F| = k THEN evict least salient entity; return definite description; add e.

6. After every choice, update descriptor and move e to the front of F.

ALGORITHM AGGREGATE(p1,p2)

1. IF p1 and p2 share an entity and have compatible tense, topic and relation THEN combine with “and”, “while”, or a subordinate clause that preserves event order.

2. ELSE output p1 as a separate sentence and retain p2 in the buffer.

3. Never aggregate if the relation is uncertain or aggregation hides a required temporal order.

Selection rule: choose the shortest referring form that uniquely identifies the intended entity for the reader, while respecting focus capacity and grammatical agreement.

A3.3 Generator Trace (1 Mark)

For a reproducible example, assume server_1 is GIVEN at F1; Ravi and suite_1 are NEW when first introduced. The microplan is converted into the following sentences. Cost convention: full name=3 words, definite description=3, pronoun=1; content words are counted separately as ordinary emitted words.

Proposition	Sentence produced	Focus after step	Aggregation buffer / flags	Running reference-form cost
F1 memory(server_1,32GB)	The server has 32 GB of memory.	[server_1]	server GIVEN; buffer flushed	3
F2 configure(Ravi,server_1)	Ravi configured it.	[Ravi, server_1]	Ravi NEW→GIVEN; server pronoun used	3+3+1=7
F3 install(Ravi,suite_1,server_1)	He installed a compiler suite	[suite_1, server_1]	Ravi evicted after suite introduced;	7+1+3+3=14

	on the server.		suite NEW→GIVEN	
F4 crash(suite_1,during=build)	It crashed during a build.	[suite_1]	suite remains for next paragraph; server dead after F3	14+1=15

Naïve baseline (full names every time): “The server has 32 GB of memory. Ravi configured the server. Ravi installed a compiler suite on the server. The compiler suite crashed during a build.” Reference-form costs: server=3, Ravi=3, server=3, Ravi=3, suite=3, server=3, suite=3 = 21 words under the given form-cost convention. The designed version uses 15 reference-form words, saving 6. A careful human might write “The server has 32 GB of memory. Ravi configured it and installed the compiler suite on it. The suite crashed during a build.” This is shorter but requires safe aggregation and clear antecedents. The saving is bought with a risk of ambiguity and the need to maintain focus correctly.

A3.4 Sacrifice Case (1 Mark)

Microplan: introduce Ravi and Ramesh, then state “Ravi called Ramesh. He approved the request.” If both men are in focus and the resolver uses recency alone, “He” may be assigned to Ramesh, although the intended referent is Ravi. The generator may produce a pronoun with two possible antecedents. A hearer model, centering transition and grammatical-role preference (subject continuity) would prevent this by choosing “Ravi” again or asking for a more explicit plan.

A4 – Improve a Weak Discourse Parser (5 Marks)

A4.1 Diagnose the Defect (1 Mark)

Weak rule output on A4 text:

Sentence pair	Weak algorithm result	Intended reading / defect
(1)→(2)	SEQUENCE or connective label absent; “It” unresolved by this algorithm.	It = lab server; crash is a RESULT of configuration/build context. Relation is not merely sequence.
(2)→(3)	SEQUENCE because no connective and nouns may not overlap.	A second server is a new entity, not the earlier server. A same head noun does not mean same

		entity.
(3)→(4)	ELABORATION if "server" overlaps, otherwise SEQUENCE.	"it" = second server; Ravi configured the new server. Must resolve entity identity, not noun overlap.
(4)→(5)	SEQUENCE if no shared noun; possibly wrong ELABORATION if shared "room".	Warm room may be an implicit cause of hardware overheating, but evidence is insufficient to assert it caused the crash.

The algorithm confuses lexical overlap with identity and relation. It misses the causal/temporal relation between configuring and crashing, and cannot distinguish the first server from a second server with the same noun.

A4.2 Entity–Discourse Graph Design (2 Marks)

Each real-world discourse referent receives one node (E1, E2…). Every mention is stored as a separate span that points to one node. A new indefinite NP ("a second server") creates a new node even if its head noun matches an existing node. A definite NP or pronoun points to an existing node only when agreement, syntax, recency and semantic compatibility support identity.

ALGORITHM UPDATE_GRAPH(sentence s)

1. Detect mentions and predicates in s.
2. FOR each mention m:
   a. IF m is indefinite/new (a second server, a faster one) THEN create a new entity node.
   b. ELSE generate compatible antecedent candidates from active entities.
   c. Resolve using agreement, syntax, recency, grammatical role and semantic type.
   d. If confidence is low, create unresolved link; do not merge nodes.
   e. Add mention node/span and edge MENTION_OF(m,E).
3. Infer discourse relation from cue, event order, causal evidence and graph structure.
4. Attach relation to an open right-frontier node; record nucleus, satellite, predecessor and successor.
5. RETURN graph and confidence annotations.

Re-introduction test: a new indefinite NP with a distinct modifier or discourse cue creates a fresh referent. Repeated definite NP/pronoun is a referring mention only if candidate identity is supported. A same noun head alone never merges entities.

A4.3 Demonstrate the Improvement (1 Mark)

A4 text graph: E1=first lab server (mentions: “lab server”, “It”); E2=second server (mentions: “A second server”, “it”); E3=Ravi; E4=vendor; E5=room; E6=module (implicit/new after vendor action).

Resolved text: (1) Ravi configured E1. (2) E1 crashed during the first build. (3) E2 arrived from the vendor. (4) Ravi configured E2 as well. (5) The room became warm. Relation set: (1)→(2) RESULT; (3)→(4) ELABORATION/SEQUENCE; (4)→(5) TEMPORAL sequence only unless thermal evidence supports cause.

T graph: E1=department; E2=server; E3=old machine; E4=lab; E5=Ravi; E6=toolchain; E7=compiler suite; E8=build; E9=fault; E10=vendor; E11=module. Links: It(2)→E2; He(4)→E5; it(4)→E2; The suite(5)→E7. “The vendor” in sentence 6 reuses E10 within the same sentence.

Required edge cases: same noun/different entity → “a second server” creates E2, not E1; cataphora → “Before he entered, Ravi checked the server” can link he to Ravi only if the construction and annotation allow cataphoric resolution; one-anaphora → “A faster one arrived” creates/reuses a server-type entity via ONE-ANAPHOR relation to the class/type, not direct identity with the old server; implicit cause → crash after build may be linked as RESULT only with supporting event evidence.

Rewrite rules: (1) if a full name repeats and the entity is uniquely in focus, replace it with a pronoun; (2) if pronoun would have a same-gender competitor, retain a definite description/name; (3) if two adjacent propositions share an argument and relation is safe, aggregate; (4) never aggregate across a paragraph/topic boundary without discourse-plan approval.

A4.4 Challenge the Design (1 Mark)

Quoted speech creates a speaker/quotation context. Mentions inside a quotation should be scoped to that reported speaker and not automatically merged with narrator commitments. A section heading opens a new discourse segment; salience may reset or be down-weighted, so old focus nodes should not be reused without evidence. Machine translation may remove gender distinctions; gender agreement can no longer be treated as a hard filter. The violated assumptions are a single discourse centre, continuous topic scope and reliable preservation of grammatical gender across languages. Keep source entity IDs and translation alignment metadata, and permit number/person plus discourse evidence to resolve gender-neutral forms.

A5 – Coherence, Global Salience and Statistical View (6 Marks)

A5.1 Opportunities and Safety (2 Marks)

Technique	Opportunity in T	Safety condition

Coreference chains	Track server, Ravi and compiler suite across sentences; use consistent names/pronouns.	Merge only when agreement, syntax and identity evidence support same referent.
Lexical chaining	Server/machine/toolchain/compiler/build form related technical topic chains.	Use semantic relation and context; repeated words can refer to different entities.
Centering	Prefer “He” for the current subject Ravi and “it” for the salient server/suite when unambiguous.	Use a pronoun only when one antecedent is clearly preferred and compatible.
Entity grid	Measure continuity of entities as subject/object/other across units.	A high grid score is a proxy, not proof of factual coherence.
Discourse extraction	Select nuclei such as buying/configuring server and crash/reporting fault.	Preserve critical satellites and causal qualifiers; do not remove safety-critical facts.
Distributional similarity	Recognise toolchain/compiler/build as related even without exact repetition.	Similarity must be domain-validated; related terms are not necessarily coreferential.

A5.2 Global Computation (2 Marks)

For each unit u, GEN(u) is the set of entity IDs introduced or made salient by u. KILL(u) is the set whose salience is ended or whose local focus is replaced by a topic shift. Forward propagation computes IN[u] and OUT[u]: IN[u] = union of OUT[p] for predecessors p; OUT[u] = GEN[u] ∪ (IN[u] − KILL[u]).

ALGORITHM FORWARD_SALIENCE(G)

1. For every unit u, initialise IN[u]=∅ and OUT[u]=GEN[u].
2. Set document entry IN[first]=∅ (no prior document entities).
3. Place all units in a work queue in document order.
4. WHILE queue is not empty:
   a. Remove u.
   b. newIN ← union of OUT[p] for all predecessors p; for document start use ∅ .
   c. newOUT ← GEN[u] ∪ (newIN − KILL[u]).

d. IF newIN or newOUT changed, update them and add successors to queue.

5. RETURN IN and OUT.

At a merge, union is used because an entity salient on either incoming path may be available. Initialise to empty at the document start. At the end, entities not killed remain in OUT. The process terminates because the entity set is finite and each update only adds information monotonically; after a finite number of additions no set changes.

A5.3 Iteration Trace (1 Mark)

Use entities E2=server, E3=old machine, E5=Ravi, E6=toolchain, E7=suite, E8=build, E9=fault, E10=vendor, E11=module. Simplified GEN/KILL sets for six units:

Unit	GEN	KILL
U1	{E1 department,E2 server,E4 lab}	∅
U2	{E3 old machine}	∅
U3	{E5 Ravi,E6 toolchain}	∅
U4	{E7 suite}	∅
U5	{E8 build,E9 fault}	∅
U6	{E10 vendor,E11 module}	∅

Iteration	IN(U1)	OUT(U1)	IN(U2)	OUT(U2)	IN(U3)	OUT(U3)	IN(U4)	OUT(U4)	IN(U5)	OUT(U5)	IN(U6)	OUT(U6)
0	∅	{E1,E2,E4}	∅	{E3}	∅	{E5,E6}	∅	{E7}	∅	{E8,E9}	∅	{E10,E11}
1	∅	{E1,E2,E4}	{E1,E2,E4}	{E1,E2,E3,E4}	{E1,E2,E3,E4}	{E1,E2,E3,E4,E5,E6}	{E1,E2,E3,E4,E5,E6}	{E1,E2,E3,E4,E5,E6,E7}	{E1,E2,E3,E4,E5,E6,E7}	{E1,E2,E3,E4,E5,E6,E7,E8,E9}	{E1,E2,E3,E4,E5,E6,E7,E8,E9}	{E1,E2,E3,E4,E5,E6,E7,E8,E9,E10,E11}

2	same	same	same	same	same	same	same	same	same	same	same	same

Two passes are shown: the first pass propagates the sets through the linear graph; the second confirms stability. A production graph with branching may require more work-queue updates.

A5.4 Backward Variant (1 Mark)

To compute what a reader must already know at each point, use backward analysis from the document end toward the start. At a merge, use intersection rather than union because a fact is guaranteed only if it is present on every incoming path. Initialise the document exit with the empty set unless an explicit reader-knowledge assumption is supplied. The result is conservative: it keeps only facts required on all paths. A summariser or translator may lose optional background links and useful inferred context because they are not guaranteed for every reader or path.

PART B – IMPLEMENTATION AND COMPUTATIONAL INVESTIGATION (25 MARKS)

Implementation note: the following compact Python reference program demonstrates sentence segmentation, simple mention detection, pronoun resolution, gold-link scoring and operation instrumentation. It is intentionally rule-based and transparent. It is not a full linguistic parser; the manual traces and limitations below must be retained in the report. Run it in Python IDLE, VS Code or Google Colab and attach screenshots of actual output.

B1 – Implement and Instrument the Discourse Analyser (7 Marks)

B1.1 Validation Tests (2 Marks)

Test	Input design	Expected behaviour / purpose
Test 1: topic shift	Two paragraphs: server report followed by library printer report; pronouns occur in each.	Paragraph break starts a new topic segment; a pronoun in paragraph 2 must not resolve to a paragraph-1 entity unless explicit evidence supports it.
Test 2: same head noun	“The first server failed. A second server arrived. It was configured.”	Create two server entities; resolve “It” to the second server by recency and agreement.

Test 3: boundary	“The compiler crashed.”	One unit, zero pronouns, no resolution links; program must not fail on an empty candidate list.

B1.2 Design versus Implementation (1 Mark)

The implementation uses punctuation-based sentence splitting and a small pronoun lexicon. It adds a pleonastic-it filter for weather/extraposition examples and treats paragraph boundaries as hard salience resets. These are practical approximations because a complete clause parser and semantic-role model are outside the compact implementation.

B1.3 Two Salience Strategies (2 Marks)

Version 1: choose the most recent agreement-compatible entity. Version 2: score candidates as 4×grammatical-role + 3×recency + 2×repeated-mention + 1×parallelism. Role matters because subjects are often continuing topics; recency matters because nearby entities are accessible; repetition indicates topic continuity; parallelism helps when consecutive clauses use the same role pattern. Hard agreement and paragraph-scope filters are applied before scoring.

B1.4 Measurement Plan (2 Marks)

Use the same annotated reports for both versions and all focus sizes. Record exact link pairs, unresolved pronouns, ambiguous pronouns, candidate-set size, word count, token count and F1. Fill measured values after running the program; do not invent results.

k	Version	Resolved / unresolved	P / R / F1	Mean candidate size	Words emitted	Tokens
1	V1	Run and record	Run and record	Run and record	Run and record	Run and record
1	V2	Run and record	Run and record	Run and record	Run and record	Run and record
2	V1	Run and record	Run and record	Run and record	Run and record	Run and record
2	V2	Run and record	Run and record	Run and record	Run and record	Run and record

3	V1	Run and record	Run and record	Run and record	Run and record	Run and record
3	V2	Run and record	Run and record	Run and record	Run and record	Run and record
Unlimited	V1	Run and record	Run and record	Run and record	Run and record	Run and record
Unlimited	V2	Run and record	Run and record	Run and record	Run and record	Run and record

Interpretation guide: if V2 improves F1 for small k, role and repetition compensate for limited memory. If the difference disappears for unlimited k, the main difficulty is ranking competing candidates rather than losing the correct antecedent from focus. If it does not disappear, the scoring features or entity typing need improvement.

B2 – Engineer an Adversarial Input (5 Marks)

Fixed focus size: k=2. Both texts contain six sentences and four pronouns.

Text A – Easy resolution

(1) The printer stopped. (2) It showed an error. (3) The technician arrived. (4) He restarted the printer. (5) The printer worked again. (6) It completed the test.

Text B – Difficult resolution

(1) Arun met Bala near the server. (2) He told him that it had failed. (3) The technician checked the printer. (4) It was beside the server. (5) He restarted it after he tested it. (6) It worked again.

Pronoun count in Text B can be standardised to four for the experiment by annotating only the four target pronouns (He, him, it, It) and rewriting sentence 5–6 without additional pronouns if needed. Use the final exact version below for equal counts:

(1) Arun met Bala near the server. (2) He told him that the server had failed. (3) The technician checked the printer. (4) It was beside the server. (5) The technician restarted the printer. (6) The printer worked again.

For strict equal-count testing, Text A and Text B must both be six sentences and exactly four pronouns. Final Text A: “The printer stopped. It showed an error. The technician arrived. He restarted it. The printer worked again. It completed the test.” Final Text B: “Arun met Bala near the server. He told him that it

had failed. The technician checked the printer. It was beside the server. The technician restarted the printer. The printer worked again." Text B has four pronouns (He, him, it, It); Text A has four (It, He, it, It).

Prediction before execution: Text B will resolve worse. First likely error is sentence 2 "He", because Arun and Bala are both compatible male antecedents. "Him" creates a second competing male candidate. Text A has a single obvious antecedent for each pronoun.

Measure	Text A prediction	Text B prediction	Measured result
Pronouns	4	4	Fill after run
Correct links	4 likely	2–3 likely	Fill after run
Unresolved / ambiguous	0	At least 1 ambiguous	Fill after run
Mean candidate-set size	Small (about 1)	Larger (about 2–3)	Fill after run
Links changed after reordering	Low	Higher	Fill after run

Reordering demonstration: place the intended antecedent immediately before the pronoun and separate competing same-gender entities. The facts remain the same, but recency and focus change. Sentence and pronoun counts alone do not measure difficulty; the key property is the number and similarity of competing antecedents plus their order and grammatical roles.

B3 – Dialogue: How Much Context Does a Conversational Agent Need? (8 Marks)

Dialogue act labels used: REQUEST, INFORM, QUESTION, ACCEPT, OFFER, THANK, ACKNOWLEDGE, CLARIFY and REPAIR. Dialogue state is a set/stack of (act, entity, value, commitment) features.

B3.1 Three Small Dialogues (2 Marks)

Dialogue / turn	Gold act	Predicted act	Appropriate response
Information: U: "What time does the library close?"	QUESTION	QUESTION	Yes – provide hours or ask date if needed.
A: "It closes at 8 PM."	INFORM	INFORM	Yes – answer supplied.
U: "Thank you."	THANK	THANK	Yes – acknowledge.
Clarification: U:	REQUEST	REQUEST	Yes – ask which item if

"Renew it for two weeks."			multiple candidates.
A: "Do you mean the NLP book?"	CLARIFY	CLARIFY	Yes – identifies missing referent.
U: "Yes."	ACCEPT	ACCEPT	Yes – confirm renewal.
Repair: U: "Renew the AI book."	REQUEST	REQUEST	Yes – process request.
U: "Sorry, I meant the NLP book."	REPAIR	REPAIR	Yes – replace prior target.
A: "I will renew the NLP book."	CONFIRM	CONFIRM	Yes – reflect corrected intent.

For supplied dialogue D, likely acts: U1 REQUEST; A1 REQUEST-INFO; U2 INFORM; A2 INFORM+OFFER; U3 ACCEPT+QUESTION; A3 INFORM+COMMITMENT; U4 ACKNOWLEDGE+THANK. The full-history and optimised state should return the same response for these turns.

B3.2 Conservative and Unsafe Cases (2 Marks)

Conservative failure: U1 asks, "Is the NLP book available?" The noun phrase NLP book is mentioned in a question, but availability is not established. If the state stores every mentioned entity as confirmed available, the agent may answer "I have reserved it for you" without checking inventory. The triggering turn is the question; the faulty path is mention detection → entity marked established/available → reservation response. The safe transformation is to store the entity as a query target, not a fact or commitment.

Unsafe transformation: U says "I need to renew three books." A asks for the ID. U provides it. If the system merges the two user informs "I need to renew three books" and "It is 2024CS118" into one vague inform, it may lose the distinction between renewal quantity and identifier. Do not drop clarification turns or merge separate commitments unless slot values and dialogue acts remain recoverable.

B3.3 Scalability Data (2 Marks)

Operation model: V1 rebuilds the state from the entire dialogue after every turn; for n turns and average f features per turn, operations are approximately $f \times n(n+1)/2$. V2 maintains a stack/window of the most recent salient acts/entities of size w; operations are approximately $f \times n \times w$. Use $w=5$ for this experiment.

Turns	Entities	Acts	Iterations V1	Iterations V2	Operations V1 (f=4)	Operations V2 (f=4,w=5)
10	5	10	10	10	220	200
20	10	20	20	20	840	400
40	20	40	40	40	3280	800
80	40	80	80	80	12960	1600

These are analytical operation counts from the stated model, not measured runtime. If the actual program uses a different feature count or window, recalculate and report the code's counters. The expected growth is quadratic for repeated full-history reconstruction and linear for a fixed-size window.

B3.4 Engineering Trade-Off (2 Marks)

The windowed state reduces repeated work and keeps recent actionable information available. Its cost is approximately O(nw), while full-history rebuilding is $O(n^2)$ under this implementation. Assume a response latency budget of 200 ms for a short library interaction on a normal laptop; use the measured operation count and runtime to decide the cut-off. I would stop rebuilding the full history once the dialogue exceeds about 20 turns, unless the user asks for a historical recap or an older entity is explicitly referenced. Keep a searchable archive so older facts can be retrieved on demand.

B4 – Final Engineering Decision (5 Marks)

B4.1 Three Optimisation Levels (2 Marks)

Level	Pipeline order	Use case / expected behaviour
Level 0 – Basic	Sentence split → direct translation → answer from current sentence only.	Lowest effort; pronouns and cross-sentence links remain unresolved. Use only for low-risk preview.
Level 1 – Fast	Sentence split → lightweight entity/reference resolution → discourse plan → surface realisation → Hindi/Tamil translation.	Default for ordinary notices; preserves main entities and event order with bounded processing.
Level 2 – Full	Document segmentation →	Use for high-impact notices,

	entity graph/coreference → discourse relations → global salience → discourse plan → surface realisation → transfer/interlingua or neural translation → target-language verification.	long reports or explicit request for careful translation.

Reference resolution must occur before translation because Hindi and Tamil may require gender, number, honorific and case choices that depend on the antecedent. Discourse planning must precede surface realisation because sentence order and aggregation must be decided before wording. Target-language verification checks that the translated text preserves participants, negation, time and instructions.

Level	Unresolved pronouns before→after	Words / sentences	Quality measure	Human adequacy	Operations
Basic	4→4 (illustrative)	source-length output	chrF/BLEU: measure on held-out set	Meaning may be unclear	1× baseline
Fast	4→1 (illustrative)	shorter, clear sentences	chrF/BLEU: measure on held-out set	Adequate if facts and roles preserved	about 3× baseline
Full	4→0 or flagged ambiguous	may aggregate or clarify	chrF/BLEU: measure on held-out set	Best expected adequacy, but human review needed	about 8× baseline

The arrows and effort multipliers are design targets, not observed experimental scores. Report actual BLEU or chrF from a held-out parallel set and a human adequacy rating (for example 1–5 for preservation of meaning, entities, time, negation and instructions). Do not report a made-up BLEU value.

B4.2 Challenge the Strategy (1 Mark)

Counterexample: “The lab door must remain closed. Do not enter until the safety officer confirms that the gas test is complete.” A full discourse planner may aggregate these into one long sentence in Hindi, making the prohibition and condition harder to notice. The fast level’s short sentence-by-sentence output may be safer. Full processing is not automatically better; safety-critical instructions should preserve sentence boundaries and explicit order.

B4.3 Production Decision Note (2 Marks)

Decision: use Level 1 (Fast) by default for ordinary notices. Use Level 2 (Full) on request or automatically for long, ambiguous or safety-critical documents. Level 0 is only a preview mode.

- Guarantees: preserve explicit proposition roles, maintain entity IDs across the document where confidence is sufficient, flag unresolved pronouns, and retain negation, time and instruction order during verification.
- Does not guarantee: perfect coreference, best natural style, correct interpretation of missing context, or a perfect translation for every dialect and domain.
- Second target language cost: share the source-language analysis and interlingua; add target-specific transfer/realisation rules and evaluation data. Separating the interlingua makes Hindi and Tamil support cheaper than building independent source-to-target systems.
- Skip or limit document-level analysis for very short, self-contained notices with no pronouns or cross-sentence relations; use a bounded window for long documents; retain full analysis for safety-critical or ambiguous content.
- Monitor: reference-link precision/recall/F1, unresolved/ambiguous rate, human adequacy, chrF/BLEU by language, latency, operation counts, error reports and failures involving names, honorifics, gender, negation and dates.

Python Reference Implementation for Part B

The following beginner-friendly program is a transparent baseline. It splits text into sentences, detects a small set of pronouns, maintains recent entity candidates, resolves using either recency or a weighted score, and calculates P/R/F1 when gold links are supplied. Extend the entity detector and add your own annotated test reports for the full rubric.

```
import re
```

```
PRONOUNS = {"he", "she", "it", "they", "him", "her", "them"}

PLEONASTIC = ["it is raining", "it is said", "it seems"]


def split_sentences(text):
    return [s.strip() for s in re.split(r"(?<=[.!?])\\s+", text) if s.strip()]


def detect_entities(sentence):
    # Small demonstration lexicon; extend for your reports.
    names = ["Ravi", "Arun", "Bala", "Leena", "vendor"]
    nouns = ["server", "machine", "suite", "compiler", "printer",
             "technician", "module", "report", "book"]
    found = []
    low = sentence.lower()
    for word in names + nouns:
        if re.search(r"\\b" + re.escape(word.lower()) + r"\\b", low):
            found.append(word.lower())
    return found


def is_pleonastic(sentence):
    s = sentence.lower()
    return any(x in s for x in PLEONASTIC)


def resolve(text, strategy="v2", k=2):
```

```
state = []
links, trace = [], []
sentences = split_sentences(text)
for si, sentence in enumerate(sentences, start=1):
    words = re.findall(r"\\b[\\w']+\\b", sentence)
    for w in words:
        token = w.lower()
        if token in PRONOUNS:
            if token == "it" and is_pleonastic(sentence):
                trace.append((si, token, [], "NON_REFERENTIAL"))
                continue
            candidates = state[-k:] if k != "unlimited" else state[:]
            scored = []
            for distance, ent in enumerate(reversed(candidates), start=0):
                recency = max(0, 3-distance)
                role = 2 if ent in sentence.lower().split()[:3] else 1
                repetition = 2 if ent in sentence.lower() else 0
                score = recency if strategy == "v1" else 4*role + 3*recency + 2*repetition
                scored.append((ent, score))
            scored.sort(key=lambda x: x[1], reverse=True)
            if scored:
                best = scored[0][0]
                links.append((si, token, best))
```

```
                trace.append((si, token, scored, best))
            else:
                trace.append((si, token, [], "UNRESOLVED"))
        else:
            for ent in detect_entities(sentence):
                if ent in state:
                    state.remove(ent)
                state.append(ent)
                if k != "unlimited" and len(state) > k:
                    state.pop(0)
    return sentences, links, trace, state


def score_links(predicted, gold):
    pset, gset = set(predicted), set(gold)
    correct = len(pset & gset)
    precision = correct / len(pset) if pset else 0
    recall = correct / len(gset) if gset else 0
    f1 = 2*precision*recall/(precision+recall) if precision+recall else 0
    return precision, recall, f1


if __name__ == "__main__":
    report = ("The department bought a new server for the lab last month. "
              "It replaced an old machine that had failed twice. "
```

```
        "Ravi configured the server with the lab toolchain. "
        "He then installed the compiler suite on it. "
        "The suite crashed when the memory filled during a build.")
sentences, links, trace, state = resolve(report, strategy="v2", k=2)
print("UNITS:")
for i, s in enumerate(sentences, 1):
    print(i, s)
print("\\nRESOLUTION TRACE:")
for item in trace:
    print(item)
print("\\nFINAL FOCUS:", state)
gold = [(2, "it", "server"), (4, "he", "ravi"), (4, "it", "server")]
print("P/R/F1:", score_links(links, gold))
```

Final Conclusion

This assignment shows that coherent language generation depends on more than grammatical sentences. A discourse analyser must identify entities, resolve references and build relations between units. A generator must maintain focus, choose clear referring expressions and aggregate only when meaning and order remain safe. Dialogue agents must distinguish questions, information, commitments and repairs. Machine translation should receive resolved meaning and a discourse plan before realisation. Instrumentation, adversarial tests and evaluation measures are necessary to determine whether an optimisation genuinely improves quality.